

Scaling up Tidal — one grammar at two scales (micro voices, macro arrangement)

Design note, 2026-08-03. Converged with AC over a design conversation. Companion to [DESIGN-tank-overhaul.md](#). Frames how much of TidalCycles Vetula's Perform cards can hold, and how the answer points at unifying them with the shell's [Triggerfish.Macro](#) arrangement layer. No code yet — this is the grammar we converge on before building.

1. The finding that drives everything

Triggerfish already runs Tidal at **two scales**, and they are converging on the *same grammar*:

- **Micro — the Vetula Perform card.** A pattern over **chord-indices within one voice**, transformed by a stack of harmonic verbs: `0 1 2 3 # voice open # arp up 4`. Atoms are *chords*; verbs are *voicing / pitch* moves.
- **Macro — [Triggerfish.Macro](#).** A lane pattern over **saved forms (glyph-tokens) across the whole rig**, transformed by per-instrument verbs: `"contemplative" # scale <"F# lydian" "G major">`. Atoms are *forms*; verbs are *whatever each instrument decides they mean*.

Both are the **same shape**: a mini-notation of atoms, then `# verb arg` layers. This is why "how much Tidal can we hold?" has one answer instead of fifty — the ceiling is a property of the shared grammar, not of either surface.

2. Where the headroom actually is (the micro ceiling)

Two independent axes, at very different maturity:

2.1 The sequence axis is already rich

The engine under `src/Tidal/**` (AST · Parse · Pattern · Eval · Notation · Chords) already gives the head segment: rests `~`, groups `[a b]`, alternation `<a b>`, euclid `a(3,8)`, replication `a!3`, speed `a*2 / a/2`, degrade `a?`. A large fraction of Tidal mini-notation is already reachable in the sequence part of a card.

2.2 The argument axis is the frontier — and the lever is NOT more verbs

Today each Perform layer takes a **scalar** argument. `parsePerfFx` reads `transpose 7` with `tokInt` — one integer. In real Tidal **everything is a pattern, including the arguments**: `# transpose "0 7 <5 3>"`.

The moment a layer's argument can itself be a mini-notation **pattern**, you get time-varying transposition, alternating voicings, euclidean gating of a verb — an expressivity explosion **with no new verbs and no new engine**, because it reuses the `<>` / sequence machinery already parsed by `src/Tidal`.

The tell: [Macro.purs](#) already did this. Its modifier argument type is `Arg = Lit String | AltArg (Array String)` — the macro args are *already* per-cycle alternation-valued. The arrangement layer is **ahead of the voice card** on exactly the axis that matters. So "scale up Tidal (micro)" is precisely: **bring pattern-valued arguments to the Perform layers**, catching the card up to what the lane already does.

2.3 More verbs is the *second* lever, not the first

Beyond scalar→pattern args, Tidal offers structural transforms we do not expose: `rev`, `off` / `superimpose`, `iter`, `palindrome`, `chunk`, `jux`, `sometimesBy`. Worth having, but *breadth*. Depth (pattern args) multiplies what the eight verbs we already have can say, and it is the move that unifies the two scales — so it comes first. **One exception jumps the queue: `sLow` — see §2.4.**

2.4 The most-missing verb is `sLow` — and it exposes a time-base assumption (AC)

The single most-wanted verb is `sLow`. You almost inevitably want a harmonic progression to unfold **much slower than a bar** — unless you are chasing bebop. Two forces make this the norm, not the exception:

- **Conditioning Odonus.** When a Vetula progression is *conditioning* Odonus (setting its harmonic frame), Odonus is running constant variations underneath that **need time to be appreciated**. A chord change every bar strobes past that motion; a chord held over 4–8 bars lets the variation speak.
- **Ambient.** Very slow progressions with **very sparse arpeggios** are a whole register the current one-bar assumption can't reach.

Three consequences for the grammar:

1. **`sLow N` becomes a first-class verb.** It exists today only *encoded as negative `rate`* (`rate -4` = 4× slower, clamped to -8). That is unintuitive and undiscoverable — nobody reaches for "negative rate" to mean "hold this longer." Promote `sLow N` (and `fast N`) to Tidal-canonical verbs; a card should say `# sLow 4`, not `# rate -4`.
2. **Rename/retire our `rate`.** In real Tidal `rate` is **sample playback speed** (a pitch control), *not* time-stretch — our signed `rate` collides with that meaning. Time belongs to `sLow/fast`; if a pitch/speed control is wanted later it can take the `rate` name honestly. (Migration: keep parsing `rate -n/rate n` as `sLow/fast` for old scenes.)
3. **The range must go large.** `sLow` wants to reach `8 · 16 · 32 · 64` bars, not cap at 8. The one-bar cycle is the *sequence's* home; `sLow` is precisely how a progression stops being a one-bar loop and becomes an arc. (Open decision 6.7: does `sLow's` cap come off entirely, and does `boxUsesSeq`/the bar-grid need to know a box is multi-bar?)

`sLow` is also a clean early win: it is a *time* transform on the whole box, independent of the pattern-arg work, so it can land first (or alongside step 1) and be felt immediately.

3. The unifying grammar — micro == macro

Once **both** layers are `atoms # verb pattern-arg`, micro and macro are the **same interpreter at two scales**, differing only in what atoms and verbs *resolve to*:

	atoms resolve to	verbs resolve to	evaluated arg feeds
micro (Vetula card)	chord indices → voicings	voicing / pitch moves	<code>applyLayer</code> on a <code>Voicing</code>
macro (shell lane)	form names → saved setups	per-instrument commands	the instrument's verb handler

The shared core is **verb + pattern-arg**, where the pattern-arg is parsed by the *same* mini-notation parser as the head sequence and **sampled per cycle** to yield the scalar (or small structure) the verb consumes. This is the Tidal way (`# speed "1 2"`), and it is the foundation that lets Vetula wire into Macro Tidal as a first-class instrument rather than a divergent dialect.

4. Proposed grammar (the thing to converge on)

4.1 A layer is **verb arg? gate?**

- **verb** — an existing keyword: `transpose · oct · rate · voice · top · bottom · arp · strum` (micro); `scale · fast · bass · ...` (macro, per-instrument).
- **arg** — either a **literal** (today's `7`, `open`, `up 4`) or a **quoted pattern** (`"0 7 <5 3>"`). Quoting is how a multi-token / alternating arg stays one field — the same device `Macro.purs` uses to quote multi-word form names.
- **gate** — the existing `every N` clause, kept as a post-hoc `When`. (Open question 6.2: whether `every` should itself become a combinator with a pattern arg.)

4.2 Canonical text (round-trip)

`printLayer` renders a literal arg bare and a pattern arg quoted: `transpose 7 · transpose "0 7 <5 3>" · arp up 4 every 2`. The pipeline stays `seq # layer # layer`, #-split (mini-notation and quoted args never contain #). The reconciliation invariant is unchanged in spirit: `parsePipeline (printPipeline box) == { seqText, stack }` — but `stack` layers now carry a `PatternArg`, not only an `Int`.

4.3 The one parser subtlety

`parseLayer` currently tokenizes on spaces and peels a trailing `every N` off the last two tokens. A quoted pattern arg contains spaces and `<>`, so the tokenizer must treat a `"..."`-delimited span as a single token **before** peeling `every`. Quote-aware tokenizing is the whole grammar cost of pattern args; everything downstream (evaluate the pattern per cycle, feed the verb) reuses `src/Tidal`.

5. What each level needs

5.1 Micro (Vetula card)

- `PerfFx` args generalise from `Int` to a `PatternArg` (a literal or a parsed mini-notation pattern over ints / shape-tokens).
- `parsePerfFx` / `printPerfFx` gain the quoted-pattern branch; `parseLayer` gets quote-aware tokenizing.
- `applyLayer` samples the arg pattern at the current cycle before applying the verb.
- The chip UI stays the second view: a pattern-valued arg shows as a compact glyph (e.g. `{0 7}`) with the text hatch as the full form — the two-views-of-one-thing rule holds.

5.2 Macro (shell lane)

Already has the parser (`tokenize` / `parseLane` / `resolveStep`) and a per-machine lane UI (task #12). Open frontier: the **modal sequencer** (task #9), the **deferred grammar** (weights `@`, replication `*`, multiple lanes, nested `<>`), and **per-instrument verb interpretation** (the parser is deliberately domain-agnostic

— each instrument must say what its verbs mean). Vetula becoming a macro instrument = its verbs (**scale**, voicing moves) resolving against a loaded form.

6. Open decisions (converge before building)

1. **Arg representation.** Reuse **src/Tidal**'s **Pattern** type for args directly, or a thin **PatternArg = Lit ... | Pat ...** that lazily parses? (Leaning: thin wrapper, parse once, sample per cycle — keeps **applyLayer** pure and the round-trip cheap.)
2. **every vs pattern-gate.** Keep **When = Always | Every n** as a separate clause, or fold gating into a pattern arg (**sometimesBy**, boolean patterns)? (Leaning: keep **every** — it reads, and it is orthogonal to the value pattern.)
3. **Shape-valued patterns.** **voice <open drop2>** and **arp <up down>** want the arg pattern to range over *shape tokens*, not ints. One **PatternArg** over strings, verb-interpreted?
4. **Quoting rule.** Always-quote pattern args (**transpose "7"** even when scalar) or quote-only-when-needed (bare **7**, quoted **"0 7"**)? (Leaning: quote-only-when-needed for readability; printer decides by whether the arg is a single literal.)
5. **How far to chase Tidal.** Ship pattern-args first (§2.2); treat the verb-breadth set (§2.3: **rev / off / iter / ...**) as a later, à-la-carte slice.
6. **Complexity budget.** The text hatch grows in *depth* (richer args), not *width* (a wall of verbs) — this is the **learn-the-chrome ≈ learn-Tidal** guardrail from **inherited-crafting-moler**. Every added verb must earn a chip. (**slow** earns its chip on sight — §2.4.)
7. **slow's ceiling and the bar grid** (§2.4). Does **slow N** cap at all, or reach arbitrarily large (32/64 bars)? And does a multi-bar box need **boxUsesSeq** / the bar-vs-beat grid (**scheduleBox**) to know its true length, or does **slow** compose cleanly on top of the existing one-bar scheduling? This is the one place **slow** touches more than a verb table.

7. Execution sequence (proposed — not started)

0. **slow / fast as first-class time verbs** (§2.4) — the quick, felt win, independent of the pattern-arg work. Promote **slow N / fast N**, migrate old **rate ±n**, lift the range for multi-bar arcs, check the bar-grid (decision 6.7). Can land first.
1. **Quote-aware parseLayer** + a **PatternArg** type; printer round-trips literal vs quoted. Reconciliation invariant re-proved. *No behaviour change yet (all args still literals).*
2. **applyLayer samples the arg pattern per cycle** — the first live pattern-arg (**transpose "0 7"**). Verify by ear.
3. **Shape-valued args** (**voice <open drop2>**, **arp <up down>**) — decision 6.3.
4. **Chip view for pattern args** — the compact glyph + text-hatch pairing.
5. **Converge micro and macro** — factor the shared **verb + pattern-arg** core so **Triggerfish.Macro** and the Vetula pipeline share one evaluator; Vetula becomes a macro instrument.
6. **Macro frontier** — modal sequencer (#9), deferred grammar (**@ / *** / multi-lane), per-instrument verb tables. Lands on the unified foundation.

8. What this is NOT

- Not a rewrite of the mini-notation engine — **src/Tidal** already has the **Pattern** type and parser; this exposes it to *arguments*.

- Not a promotion of the chip UI to a second language — chips stay the discoverable view of the one text pipeline.
- Not the macro sequencer itself — that is task #9, which this note sets the foundation for but does not do.